

Graphical Abstract

SENTRY: A Layered Silent Data Corruption Detection Technique Based on Deterministic Transaction Execution.

Yu Yan, Sijia Cheng, Yuqi Zhang, Zhiyu Dai, Hongzhi Wang

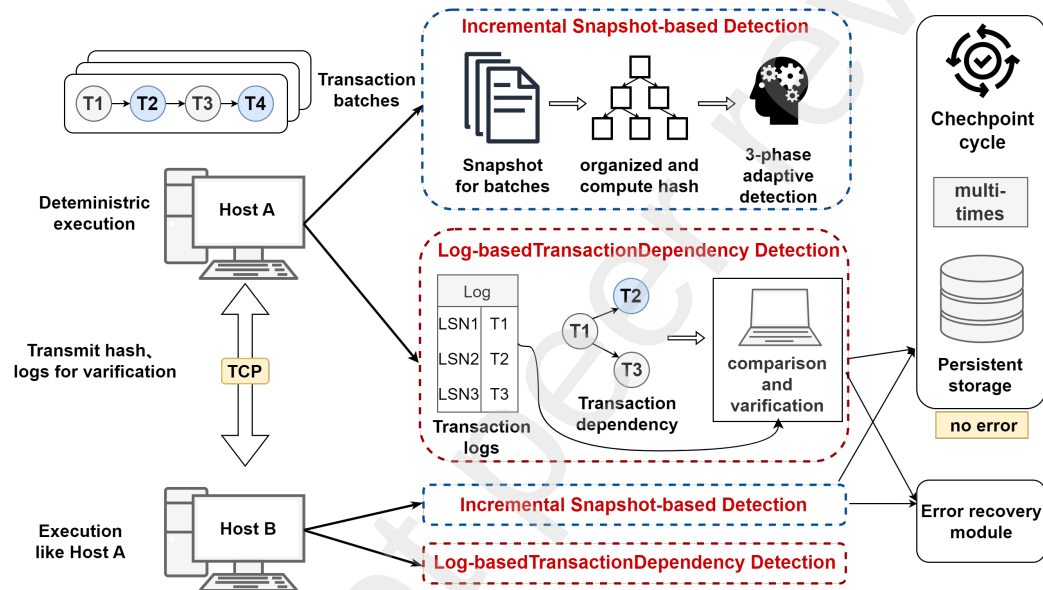


Figure 1: The framework of the proposed detection technique

Highlights

SENTRY: A Layered Silent Data Corruption Detection Technique Based on Deterministic Transaction Execution.

Yu Yan, Sijia Cheng, Yuqi Zhang, Zhiyu Dai, Hongzhi Wang

- Introduces SENTRY, a database-tailored silent data corruption (SDC) detection framework combining incremental snapshot verification with transaction dependency analysis, ensuring data integrity in cloud environments.
- Proposes a high-frequency incremental snapshot mechanism with node-level selective hashing and layered hash tree structure for fast, sub-second consistency verification across distributed hosts.
- Develops a log-based transaction dependency detection technique that analyzes transaction execution to precisely locate and identify SDC-affected transactions.
- Achieves a dual-layer detection architecture that balances coverage, accuracy, and performance, supporting both dual-primary and multi-primary database configurations.

SENTRY: A Layered Silent Data Corruption Detection Technique Based on Deterministic Transaction Execution.

Yu Yan, Sijia Cheng, Yuqi Zhang, Zhiyu Dai, Hongzhi Wang

*Massive Data Computing Lab, Harbin Institute of Technology, No. 92, Xidazhi Street,
Nangang District, Harbin, 150006, Heilongjiang, China*

Abstract

In cloud computing environments, database systems usually run on general-purpose hardware, which is susceptible to silent data corruption (SDC). Unlike conventional errors, SDC does not immediately trigger system crashes or alarms and lacks obvious signs, which may lead to irreversible data corruption over time, seriously threatening system reliability. Existing SDC detection technologies face a fundamental trade-off: hardware-based solutions, while effective, impose prohibitive costs and scalability constraints; software-based approaches suffer from poor timeliness, limited generality, and inadequate systematic design, with some methods exhibiting platform dependencies that restrict their applicability.

To address these critical challenges, this paper presents **SENTRY**, the first database-tailored SDC detection framework that integrates snapshot-based incremental verification with log-level transaction dependency checking. This dual-layer architecture provides a detection mechanism from both transaction execution and database state perspectives, ensuring database consistency while enabling rapid identification of transactions where SDC occurs, creating a comprehensive data integrity protection mechanism specifically designed for database workloads. Extensive experiments show that our method can efficiently detect inconsistencies caused by SDC during system operation and precisely locate the erroneous transactions, while improving the detection rate by approximately **20%** and the detection efficiency by approximately **73.03%** compared to traditional software approaches, thereby verifying both the accuracy and efficiency of our approach.

Keywords: silent data corruption detection, deterministic transaction execution, layered detection, transaction dependency

1. Introduction

In the context of rapid global digital transformation and the swift advancement of cloud computing, industries are placing unprecedented demands on the real-time performance, reliability, and scalability of database systems. The scale of data that enterprises need to process has increased from terabytes (TB) to petabytes (PB) and even exabytes (EB), posing significant challenges to the performance and stability of database systems [17, 18, 37]. Database systems form the backbone of modern digital infrastructure, underpinning financial transactions [29], healthcare records [27], e-commerce platforms [28] and so on. Any compromise to database integrity can cascade through entire organizational ecosystems, potentially affecting millions of users and resulting in irreparable damage to business operations and public trust. Given the substantial hardware costs associated with processing such massive data, database systems in cloud computing environments typically operate on clusters of commodity hardware rather than expensive, high-reliability specialized equipment.

However, in production environment, factors such as high-energy particle radiation and unstable power supply may induce hardware faults that cause bit flips. In many cases, such faults result in SDC. SDC is a type of data error caused by hardware faults, firmware defects, or latent device failures that modify stored or computed values without triggering immediate detection or explicit failure signals. SDC typically occurs with low probability and is difficult to predict, can originate from various hardware or firmware sources without unified detection mechanisms [1], and does not manifest obvious external symptoms (e.g., system crashes). If left undetected and unrecovered, such corruption can spread and cause severe consequences [2].

In 2018, Tencent Cloud disabled data integrity checks during large-scale migration for efficiency gains, but silent disk errors went undetected, allowing corrupted data to propagate system-wide and ultimately resulting in massive irreversible user data loss [3]. In 2022, Alibaba Cloud reported sporadic checksum errors in certain ECS instances, causing false data corruption alarms and performance degradation [4]. Google Cloud disclosed floating-point computation errors in scientific workloads due to processor

faults, leading to incorrect analysis results [5]. Meta (formerly Facebook) experienced firmware defects in NVMe SSDs that misreported file metadata, triggering cascading deletions in distributed storage [6]. These cases underscore that addressing SDC in commodity hardware is crucial for ensuring data integrity and system reliability. SDC often occurs without any overt signs across various systems, but its potential risks and consequences are profoundly serious and must not be underestimated. As enterprises increasingly migrate critical workloads to cloud environments and rely more heavily on data-driven decision making, the potential impact of undetected silent data corruption grows exponentially, making robust detection mechanisms absolutely essential for maintaining system credibility and operational continuity.

Existing solutions primarily address the problem at the hardware level by incorporating verification mechanisms into chips to fundamentally prevent the occurrence of silent errors. However, such approaches incur high research and development costs and require replacing the chips of existing machines, resulting in substantial deployment expenses and limited scalability. Existing software-level solutions mainly include application-level detection, I/O and data transfer detection, compiler-based instrumentation and machine learning workload protection. Some existing software-level methods rely on externally observable symptoms that compromise transactional integrity; other mechanisms lack database semantic awareness or require specialized toolchains and extensive code modifications, making them unsuitable for database systems.

These fundamental incompatibilities reveal three critical challenges in designing SDC detection specifically for database systems.

(1) Excessive detection latency incompatible with transaction processing requirements. Existing software methods suffer from inherent delays that violate database performance constraints. Most approaches adopt post-hoc detection paradigms, identifying errors only after they manifest as observable anomalies rather than preventing corruption during execution. Application-level methods depend on external symptoms that emerge only after error propagation, while compiler-based approaches detect at instruction level but fail to capture transaction-level semantics. Database SDC detection must maintain sub-second response times to support high-throughput operations.

(2) Inability to simultaneously achieve comprehensive coverage, high accuracy, and acceptable performance. Current detection methods face fundamental trade-offs that compromise database requirements. I/O protec-

tion mechanisms provide broad coverage during data transfers but operate with coarse granularity that generates false positives from legitimate database state variations. Compiler-based instrumentation delivers fine-grained, instruction-level detection but imposes prohibitive performance overhead unsuitable for high-throughput systems. Machine learning approaches offer adaptive detection capabilities but require training phases and statistical validation that conflict with databases' need for deterministic, real-time transaction validation.

(3) Complexity of deterministic execution validation in distributed environments. Modern database systems exhibit inherent complexities that existing SDC detection algorithms cannot adequately address. High-concurrency transaction execution creates intricate operation interleavings that obscure error source isolation and complicate the distinction between legitimate concurrency effects and corruption-induced anomalies. ACID transaction properties further compound detection complexity by requiring validation across atomicity, consistency, isolation, and durability guarantees. Additionally, temporal dynamics in database workloads introduce complex timing dependencies that significantly challenge the design of effective, real-time detection mechanisms.

To address the difficulties above of detecting SDC in database system, this paper introduces SENTRY, a groundbreaking detection technique that fundamentally reimagines SDC detection by eliminating the traditional trade-off between detection accuracy and detection efficiency. The proposed strategy bases on deterministic transaction execution and supports both dual-primary and multi-primary architectures. It leverages dual verification mechanisms, including incremental snapshot checking and transaction dependency detection, to ensure that SDC can be detected promptly upon occurrence, enabling the system to respond on time. This approach significantly enhances the fault tolerance of cloud database systems operating on commodity hardware, effectively reducing the risks of data loss, inconsistency, and system crashes caused by SDC. By improving system reliability and efficiency, the method contributes to the continued advancement of cloud computing services.

The contributions of this paper represent the first comprehensive approach to database-specific SDC detection and are summarized as follows:

(1) Integrated detection architecture achieving optimal coverage-accuracy-performance balance. We design a dual-layer detection system that simultaneously addresses comprehensive coverage, high accuracy, and acceptable performance. The incremental snapshot layer provides broad coverage with

fine-grained detection granularity through selective hashing. The log-based transaction dependency detection mechanism can identify SDC occurring during transaction scheduling and locate the specific transactions in which the errors occur.

(2) Incremental snapshot-based detection technique: We propose a high-frequency incremental snapshot detection mechanism that generates snapshots periodically during batch execution. These snapshots are organized into a layered hash tree-based and introduce a node-level selective hashing strategy, enabling fast snapshot comparison across hosts, achieving sub-second, rapid and timely detection.

(3) Log-Based Transaction Dependency Detection Technique: We propose a novel transaction-level verification approach that analyzes transaction logs to perform transaction dependency detection. Our method enables precise diagnosis at transaction granularity, effectively pinpointing the specific transactions causing inconsistencies and determining whether errors originated in the scheduling layer.

(4) We evaluate our approach on a dual-host architecture in a deterministic transaction execution system, and extensive experimental results demonstrate that our method achieves rapid and accurate detection of SDC during transaction execution. Compared with existing approaches, it provides a significant improvement in error detection capability, achieving an approximately **20%** increase in detection rate and about a **73.03%** improvement in detection efficiency.

Section 4 describes the timely detection techniques employed at the incremental snapshot layer, while Section 5 elaborates on fine-grained asynchronous consistency detection at the log level. Section 6 presents the experimental results and comparative analysis. Finally, Section 7 summarizes the main contributions and outlines directions for future work.

2. Related work

Existing studies on SDC focus primarily on two complementary aspects: software-level protection techniques and hardware-centric detection mechanisms. However, both categories face fundamental limitations when applied to deterministic database systems: hardware-based methods incur prohibitive deployment costs and infrastructure dependencies, while software-based methods suffer from detection latency, lack database semantic awareness, or require extensive system modifications. This section reviews repre-

sentative research efforts in each area and positions our approach within the broader landscape of SDC detection and mitigation.

2.1. Software-level Detection Mechanisms

Software-level methods detect SDC by instrumenting application code, leveraging semantic invariants, duplicating computations selectively, or applying algorithm-specific resilience techniques. These approaches align detection mechanisms with application logic rather than requiring specialized hardware support, offering greater deployment flexibility and cost-effectiveness. However, software-level approaches face fundamental challenges in real-world deployment scenarios. First, they suffer from detection latency: most methods only identify corruption after it has propagated through multiple computational layers or even persisted to storage, making timely recovery difficult. Second, they struggle with the performance-accuracy tradeoff: lightweight methods often miss subtle corruptions to maintain acceptable performance, while comprehensive detection mechanisms introduce prohibitive computational overhead. Many existing software technologies also require extensive application modifications or specialized toolchains.

These factors collectively constrain their applicability to database systems. Application-level methods rely on externally observable symptoms that compromise transactional integrity; I/O protection mechanisms lack database semantic awareness; compiler-based approaches require specialized toolchains and extensive code modifications; and machine learning-oriented methods assume iterative workloads incompatible with database query-response patterns.

Application-level Detection. Dutta et al. developed a system that detects hardware-induced SDC through application-layer logs, crash signals, and anomalous behavior analysis, achieving high detection accuracy in real-world data center deployments [13]. Their method demonstrates effective detection using existing telemetry signals without requiring hardware changes. Berrocal et al. proposed a partial replication strategy based on the observation that data variability differs across processes [16]. They selectively replicate processes with poor detector performance and introduce evaluation metrics based on undetected corruption probability, offering a more nuanced understanding of detection efficacy. This adaptive replication balances coverage and cost by targeting high-risk segments.

I/O and data transfer protection. Charyyev et al. proposed RIVA, a mechanism that enhances SDC detection during data transfer through cache

flushing, direct disk reads, and parallel verification [14]. Recent work on exabyte-scale database systems [26] highlights the importance of software-level end-to-end checking. This study proposes semantic-aware strategies such as end-to-end checksums, fail-stop invariants for complex SQL operations, and prioritization of computation-intensive components.

Compiler-based instrumentation. Recent advances in compiler-based instrumentation have emerged as promising directions for SDC protection. The differential checksum framework automatically inserts lightweight checksums into C/C++ data structures, achieving up to 95% SDC reduction compared to traditional checksum methods that often fail under transient faults [20]. FastFlip enables scalable fault injection analysis during iterative software development, reducing analysis cost by up to $17.2\times$ while guiding selective protection of the most vulnerable instructions [21]. Yang et al. used partial fault injection and support vector machines to train models that accurately identify critical instructions vulnerable to SDC, thereby reducing protection overhead [15]. This approach enables precise protection by focusing only on the most vulnerable code paths.

Machine learning workload protection. The increasing importance of machine learning workloads has prompted specialized SDC detection approaches. TrainCheck automatically infers invariants during deep learning training, catching silent errors in a single iteration and revealing new bugs in widely used training libraries [24]. ATTNChecker introduces algorithm-based fault tolerance specifically for Transformer attention mechanisms, detecting and correcting extreme numerical faults with only 7% runtime overhead [25]. A recent comprehensive study on large language model training revealed that SDC may not only perturb gradients but also shift convergence trajectories, highlighting the need for application-level semantic awareness in detection mechanisms [23].

2.2. Hardware-based Detection Mechanisms

Hardware-level approaches detect and mitigate SDC by directly deploying redundancy, integrity checking, or architectural monitoring mechanisms at the hardware layer to capture errors at their source. While these methods generally provide strong fault coverage and rapid detection capabilities, their deployment requires substantial costs and depends heavily on specific hardware configurations, making them difficult to scale and unsuitable for silent error detection in cloud database systems.

Representative works include Oracle’s comprehensive end-to-end protection strategy that synchronously generates integrity metadata during data writes and verifies this metadata throughout the entire data path from application layer to disk [10]. Makaram et al. proposed a link-level data protection scheme integrating encryption with message authentication codes (MACs) to prevent SDC during data transmission [11]. Through extensive monitoring in hyperscale data centers, Dixit et al. classified various types of silicon defects and proposed integrated hardware-software protection strategies [2]. Recent advances have explored lightweight approaches, including Zhang et al.’s heterogeneous CPU-based replicated execution [12] and ParaVerser’s co-design methodology that repurposes small cores as verification engines [22]. Despite these innovations, hardware-level solutions remain fundamentally unsuitable for software-layer SDC detection in distributed database environments due to their inherent infrastructure dependencies and scalability constraints.

3. Overview

3.1. Background

3.1.1. Deterministic Transaction Execution

Deterministic transaction execution eliminates runtime uncertainty by pre-scheduling transaction execution order at the system level, ensuring that identical inputs produce consistent outcomes across different nodes regardless of local execution variations [19]. This approach shifts concurrency control from execution time to the scheduling phase, avoiding traditional issues such as locking, blocking, and deadlocks while enabling state machine replication for distributed systems [9]. Representative deterministic systems like Calvin [9] and Aria [8] achieve high throughput by decoupling transaction scheduling from execution, allowing pipeline-style processing and parallel execution of non-conflicting transactions.

The deterministic execution paradigm offers multiple advantages for detecting SDC in database systems. Given the same input, all database replicas are guaranteed to produce identical results. By comparing the final states of different replicas, we can comprehensively verify whether data corruption has occurred across the system. When inconsistencies are observed, they cannot be attributed to normal variations in concurrent execution but must instead indicate a violation of data integrity. This greatly simplifies error analysis, allowing us to directly focus on identifying genuine corruption. These properties provide a solid theoretical foundation for our detection approach,

enabling both rapid error discovery and precise fault localization without disrupting normal database operations.

3.1.2. Transaction log structure for SDC detection

Transaction logs serve as the foundation of database systems, recording all modifications to ensure durability, consistency, and recoverability [30, 31]. For SDC detection, transaction logs provide essential fine-grained operational records that enable corruption pattern analysis and cross-replica consistency validation. Existing logging structures include Write-Ahead Logging (WAL) with sequential append-only organization [32], tree-based structures such as B+ trees and LSM-trees offering indexed access [33, 34], and distributed logging systems supporting multi-node replication [35, 7]. The choice of logging structure directly determines the feasibility and efficiency of the fine-grained consistency checking essential for comprehensive SDC detection, making robust log organization a critical foundation for any database corruption detection mechanism.

To support efficient SDC detection operations, our approach employs a tree-based transaction logging structure organized by Log Sequence Numbers (LSNs) with batched flushing capabilities. This design choice balances write performance with query efficiency, enabling both sequential log generation during normal operation and efficient range queries during consistency validation phases. The hierarchical organization allows for parallel log processing across different tree branches, supporting the high-throughput requirements of deterministic transaction systems while maintaining the detailed operational records necessary for accurate corruption detection.

The tree structure incorporates internal node buffering to batch multiple log entries before propagation, reducing insertion overhead and tree maintenance costs [36]. Since LSNs are monotonically increasing, most insertions occur at the tree’s rightmost boundary, minimizing rebalancing operations while facilitating efficient chronological access patterns. This logging infrastructure serves as the foundation for our subsequent snapshot generation and consistency validation mechanisms, providing the detailed operational history required for comprehensive SDC detection without introducing significant performance penalties to the underlying deterministic transaction system.

3.2. Problem Definition

System Model and Assumptions. We consider a distributed deterministic database system consisting of n nodes, where each node maintains a database replica and executes transactions deterministically using identical scheduling algorithms to ensure consistent results for identical inputs.

Besides, the system assumes the existence of a global scheduler that maintains consistent transaction dependency relationships across all replicas. The system operates in batch processing mode, organizing transactions into batches that are processed in the same order across nodes, and runs within checkpoint cycles where each cycle contains multiple batch execution periods before data persistence, with detection and validation performed prior to persistence. The system faces the threat of hardware faults that may cause SDC, which modifies data values without triggering explicit error signals and can occur during computation, memory access, or data transmission. The network environment is assumed to be eventually reliable for exchanging detection metadata between nodes, though temporary partitions may occur without affecting the correctness of the detection algorithm.

Problem Statement. Given the deterministic execution guarantee, any divergence in database states or intermediate results across replica nodes indicates the occurrence of silent data corruption. The core challenge is to efficiently detect such divergences before corrupted data is persisted to storage, while maintaining minimal performance overhead on normal transaction processing. This requires both rapid detection capabilities for immediate response and comprehensive validation mechanisms for accurate corruption localization.

(1) Input: Transaction log sequences from all nodes within checkpoint cycle C_j (containing metadata such as transaction ID, table ID, key-value pairs, and so on), incremental snapshots taken at predefined intervals, and the checkpoint period length.

(2) Output: Inconsistency alerts identifying conflicting nodes and detection cycles, the total count of inconsistencies detected within the cycle, and specific corruption location information to support recovery operations.

(3) Objective: Design an efficient SDC detection mechanism that detects inter-node inconsistencies before data persistence, minimizes detection latency while maintaining high accuracy and low performance overhead, and supports both rapid detection and comprehensive validation.

3.3. Method overview

SDC is a type of undetectable fault that may lead to data inconsistency or even system crashes, posing a serious threat to the reliability and stability of database systems, especially when running on general-purpose hardware. Targeting deterministic transaction execution scenarios, this paper proposes a layered SDC detection technique based on the software level. Without relying on specialized hardware, our approach achieves efficient identification and rapid recovery from SDC through a software-based mechanism. The proposed technique is applicable to dual-host and multi-host environments. Our study focuses on the dual-host deterministic transaction execution scenario. Compared with multi-host architectures, the dual-host design reduces computational hardware requirements and avoids the complexity of majority voting and recovery synchronization.

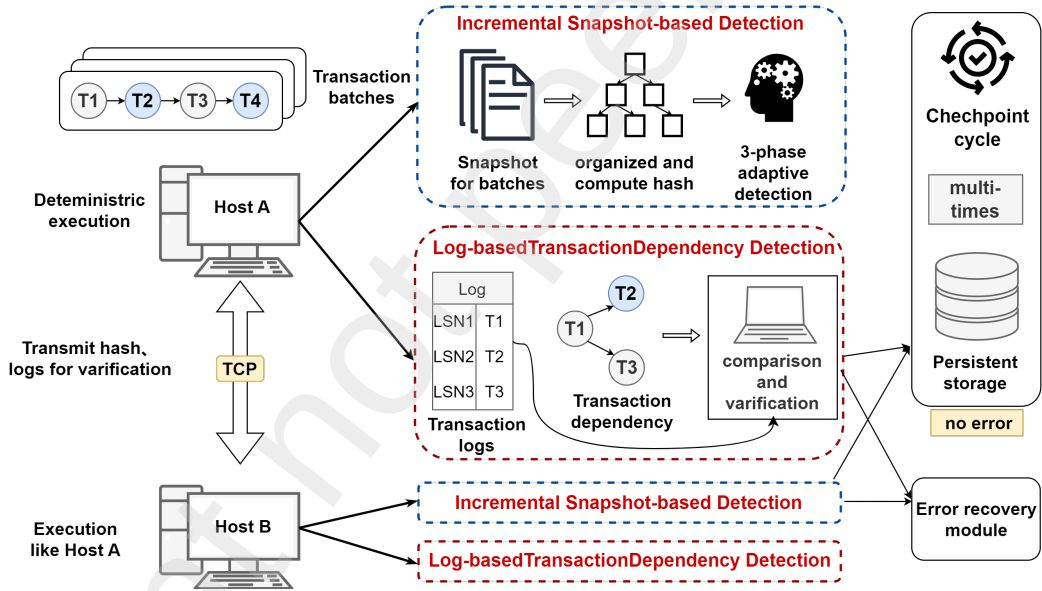


Figure 2: The framework of the proposed detection technique

The proposed method comprises two detection layers. The first layer performs incremental snapshot-based validation. After each transaction batch, snapshots are asynchronously generated and organized using a layered hash tree. In this structure, we designed different levels of nodes to display different hash computation methods. By combining partition-level hash comparisons with breadth-first traversal, our methods achieve subsecond consistency

checks. The second layer introduces fine-grained detection at the transaction log level. Before data persistence, transaction dependencies are analyzed, and logs are globally sorted based on unified transaction IDs. This resolves execution-order mismatches across hosts and ensures precise consistency verification at the log level. The two-layer mechanism significantly improves detection sensitivity and accuracy.

An overview of the proposed detection framework is shown in Figure 2. The first-layer detection mechanism of our method quickly verifies whether the database states of the two hosts remain consistent after each batch of transactions. If an inconsistency is detected, the system promptly identifies it as an error and triggers recovery policies, ensuring a timely response. However, even when the states appear consistent, errors cannot be completely ruled out, as some may be masked by subsequent writes. To address this, the second layer introduces transaction dependency detection based on transaction logs, enabling comprehensive identification of SDC during transaction execution.

Together, these two layers complement each other to ensure both transaction execution consistency and overall database correctness, providing temporal separation that balances rapid response capabilities with comprehensive validation accuracy. Our approach supports a wide range of practical application scenarios, providing a plug-and-play fault-tolerant mechanism for cloud database systems.

4. Incremental Snapshot-Based Detection Technique

This section presents our adaptation of incremental snapshot detection for database SDC scenarios. The system assumes that transactions are executed in batches. Within each persistence cycle, multiple batches are processed sequentially. After the completion of each batch, the system immediately generates an incremental snapshot and performs partition-level comparisons to verify data consistency across hosts. Compared with traditional approaches that only verify consistency at checkpoints, incremental snapshot detection enables the timely identification of potential data inconsistencies after each batch, significantly improving the sensitivity and responsiveness of fault detection and preventing error accumulation until the checkpoint stage.

4.1. Partitioned Snapshot Hash Verification Based on a Transaction-Aware Hierarchical Indexing Structure

The generation of incremental snapshots is implemented using a log query and merge approach. Assuming that transactions are executed in batches, once a batch completes, the system automatically triggers the generation of an incremental snapshot for that batch. Within a full checkpoint interval, multiple batches can be executed, and multiple incremental snapshots can be generated for comparison. The implementation process is as follows: the system first reads the new transaction logs generated since the last incremental snapshot, based on the log sequence number (LSN). For example, if the LSN of the last snapshot ends at 100, the current snapshot starts from LSN = 101. To efficiently organize this snapshot data, we employ a transaction-aware hierarchical indexing structure.

This structure organizes snapshot entries in a four-level hierarchy. The first level serves as the root node. The second level stores table identifiers, the third level maintains partition identifiers, the fourth level contains the keys of modified fields, and the leaf nodes store the corresponding updated values. Each transaction log entry generates one snapshot data record within this hierarchical organization. Before insertion, the system queries the hierarchical structure to check whether a specific key path already exists. If it does, the value in the leaf node is updated; otherwise, new nodes are created and inserted into the structure accordingly.

After the incremental snapshot is created, the system computes hash values at the partition granularity to facilitate efficient comparison. Specifically, for each distinct partition ID, the corresponding subtree in the hierarchical structure is individually hashed. To ensure that the hash values accurately reflect the actual data content and are not affected by variations in storage order, the data entries within each partition are sorted by key prior to hashing. Then, a hash function is applied to the sorted content.

During comparison, only the hash values of the same partition from different hosts need to be checked. If a mismatch is found, it indicates that a SDC occurred during this interval, and the system must initiate fault tolerance mechanisms. If the hash values match, the database states are considered temporarily consistent; however, this does not guarantee that no errors occurred, so a finer-grained log-level verification is still required.

The pseudocode for this process is shown in Algorithm 1.

Lines 1–2 construct the transaction-aware hierarchical indexing structure;

Algorithm 1: Partition-level snapshot hash verification

Input: Two transaction logs $\log A$ and $\log B$
Output: Partition-level comparison result

```
1  $treeA \leftarrow BuildTransactionAwareHierarchy(\log A);$ 
2  $treeB \leftarrow BuildTransactionAwareHierarchy(\log B);$ 
3  $hashesA \leftarrow ComputePartitionHashes(treeA);$ 
4  $hashesB \leftarrow ComputePartitionHashes(treeB);$ 
  /* ComputePartitionHashes returns a map keyed by
    (table_id, partition_id) */
  /* with hashes computed over canonical serialization
    (keys sorted) */
5  $differences \leftarrow \{partitions : \{\}\};$ 
6  $all\_keys \leftarrow union\_keys(hashesA, hashesB);$ 
7 foreach (table_id, partition_id)  $\in all\_keys$  do
8    $hashA \leftarrow hashesA.get((table\_id, partition\_id), None);$ 
9    $hashB \leftarrow hashesB.get((table\_id, partition\_id), None);$ 
10  if  $hashA \neq hashB$  then
11     $differences[partitions][(table\_id, partition\_id)] \leftarrow$ 
       $(hashA, hashB);$ 
      /* hashA or hashB may be None indicating missing
        partition */
12  end
13 end
14 if  $differences[partitions] = \{\}$  then
15   return All partition hashes are consistent;
16 end
17 else
18   return  $differences;$ 
      /* Structured list of mismatched or missing partitions
        */
19 end
```

Lines 3–4 compute hash values for each partition; Lines 6–21 iterate through all partition hashes for comparison and output the comparison result.

Assume the total number of nodes is n , and there are p partitions, with the number of nodes in partition i denoted by m_i , where $i = 1, 2, \dots, p$. The time complexity of the hash computation (primarily due to sorting) is: $O(\sum_{i=1}^p m_i \log m_i)$. The construction of the transaction-aware hierarchical structure requires creating one node for each value in the worst case, resulting in a time and space complexity of $O(n)$. The hash comparison step involves scanning all hash values, yielding a time complexity of $O(p)$.

Although the above method can effectively detect data inconsistencies, it still exhibits limitations when applied to large-scale systems. Specifically, it requires computing and comparing the hash values of all partitions one by one; even if the majority of data remains consistent, redundant computational overhead is unavoidable. Moreover, once an inconsistency is detected, the localization granularity is restricted to the partition level, making it difficult to further narrow down the scope to a specific table or finer level of detail.

4.2. *Highly efficient Snapshot Validation Via a Multi-Dimensional Hash Embedding Architecture*

To address the limitations of the aforementioned approach, this section introduces an optimized method based on structural improvements. The key enhancements of this method lie in the introduction of multi-level hash embedding, the adoption of a hierarchical comparison strategy, and the provision of fine-grained localization capability.

We propose an efficient snapshot verification technique leveraging a multi-dimensional hash embedding architecture. The core idea of the method is to explicitly compute hash values for each node based on the type of snapshot data and embed these hash values into the corresponding levels of the tree. This enables rapid identification and localization of inconsistent regions during snapshot consistency checks without requiring a full scan of all data nodes. The proposed hierarchical hash tree is organized into four levels, from top to bottom:

- (1) Root Node: Stores the hash values of table-level nodes.
- (2) Table Node: Each node key represents a table ID, and the value is the hash of its corresponding partition-level nodes.
- (3) Partition Node: Each node key represents a partition ID, and the value is the hash of its associated key-value pairs.

(4)Key-Value Node: Stores the keys and corresponding values from the transaction logs.

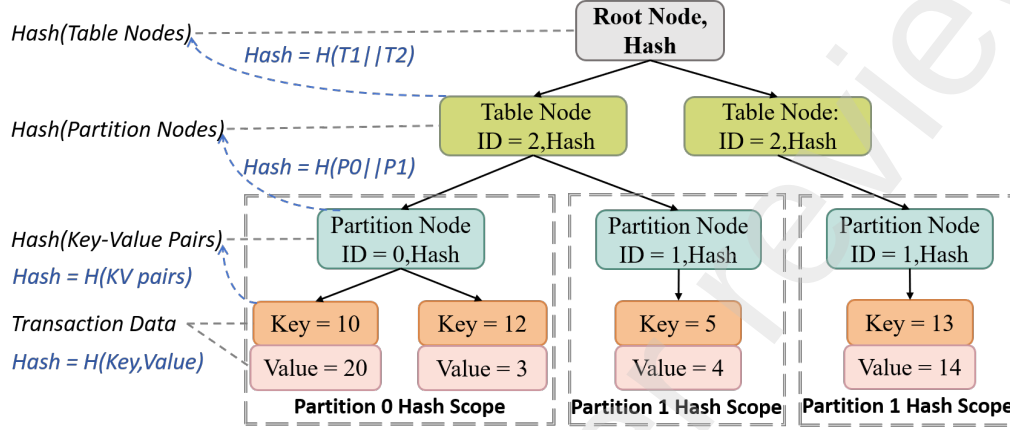


Figure 3: The multi-dimensional hash embedding architecture of incremental snapshot

After all key-value nodes are stored, the system calculates the hash of each key-value node using the SHA-256 algorithm, and stores the resulting hash in the corresponding partition-level node. Then, the hash of each partition node is calculated and stored in the table-level node, followed by computing the table node hash and storing it in the root node. Since partition and table nodes contain relatively small amounts of data and already include hashes of their child nodes, a short hash function can be applied at these levels to improve computational efficiency. An example of the multi-dimensional hash embedding architecture is shown in Figure 3 below.

During comparison between incremental snapshots from two hosts, the process begins with a three-phase adaptive validation strategy starting from the root hash. Phase 1 is a root-level early termination check. If the root hashes of both trees match, the entire snapshot trees are considered consistent and the comparison terminates. Otherwise, the algorithm traverses down to phase 2(adaptive hierarchical traversal check), comparing the hash values of each table node. This process repeats recursively until a mismatched partition is found. Subtrees with identical hashes are skipped, thereby significantly improving comparison efficiency. Phase 3 returns the structured inconsistency classification report. The pseudocode for this process is shown in Algorithm 2.

Lines 1–3 retrieve the hierarchical hash structures from both snapshot

Algorithm 2: Three-Phase Adaptive Detection

Input: Hierarchical snapshot structures $tree_A, tree_B$

Output: Structured inconsistency report with granular localization

```
1  $hashMap_A \leftarrow tree_A.get\_all\_hashes()$ ;
2  $hashMap_B \leftarrow tree_B.get\_all\_hashes()$ ;
3  $inconsistencies \leftarrow \{\}$  /* Multi-level inconsistency tracker
   */
   /* Phase 1: Root-level early termination check */
4 if  $hashMap_A[root] = hashMap_B[root]$  then
5 |   return  $\emptyset$  /* Complete consistency, terminate early */
6 end
   /* Phase 2: Adaptive hierarchical traversal check */
7  $suspects \leftarrow \emptyset$ ;
8 foreach  $tid \in hashMap_A[tables].keys() \cup hashMap_B[tables].keys()$ 
   do
9 |   if  $hashMap_A[tables][tid].hash \neq hashMap_B[tables][tid].hash$ 
   then
10 |    $suspects \leftarrow suspects \cup \{tid\}$ ;
   /* Granular partition-level analysis for suspect
      table */
11 |    $partitionDiffs \leftarrow \emptyset$ ;
12 |   foreach  $pid \in hashMap_A[tables][tid].partitions.keys()$  do
13 |   |    $hashA \leftarrow hashMap_A[tables][tid].partitions[pid]$ ;
14 |   |    $hashB \leftarrow hashMap_B[tables][tid].partitions[pid]$ ;
15 |   |   if  $hashA \neq hashB$  then
16 |   |   |    $partitionDiffs[pid] \leftarrow (hashA, hashB)$ ;
17 |   |   end
18 |   end
19 |    $inconsistencies[tid] \leftarrow partitionDiffs$ ;
20 | end
21 end
   /* Phase 3: Return structured inconsistency
      classification */
22 return  $inconsistencies$  /* Hierarchical inconsistency report
   */
```

trees and initialize the inconsistency tracking mechanism. Lines 4–6 implement the root-level early termination optimization. If the root hashes are identical, this indicates complete consistency across both snapshots, allowing the algorithm to terminate immediately without further comparison. Lines 8–21 execute the adaptive hierarchical traversal check. The algorithm first identifies suspect tables by comparing table-level hashes from both snapshots. For each table where hash mismatches are detected, indicating potential SDC, the algorithm immediately performs granular partition-level analysis. This selective approach avoids unnecessary comparisons of consistent tables while focusing computational resources on areas where inconsistencies exist. Within each suspect table, partition-level hashes are compared to precisely localize the corrupted regions. Line 22 returns the structured inconsistency report, providing hierarchical classification of detected discrepancies at both table and partition levels. Assume there are n total nodes, p partitions, and q tables. The number of nodes in each partition is denoted by m_i , where $i = 1, 2, \dots, p$, and the number of partitions in each table is denoted by p_j , where $j = 1, 2, \dots, q$. The time complexity of hash computation, which is primarily due to the sorting operations within each layer, is given by:

$$O(n \log n + p \log p + q \log q) \quad (1)$$

where $n = \sum_{i=1}^p m_i$ is the total number of nodes, $p = \sum_{j=1}^q p_j$ is the total number of partitions, and q is the total number of tables. In the worst-case scenario, constructing the tree requires creating one node per value, resulting in a time complexity of $O(n)$ and a space complexity of $O(n + p + q)$. During the comparison phase, a layered, on-demand hash comparison strategy is adopted, enabling the verification to be conducted selectively across levels. As a result, the comparison time complexity is reduced to $O(\log p)$.

5. Log-Based Transaction Dependency Detection Technique

5.1. Design Motivation

The snapshot-based detection operates on execution results at batch granularity, providing fast problem discovery through state comparisons. However, it suffers from two critical limitations that necessitate a complementary mechanism operating at transaction-level granularity. First, snapshot comparison lacks fault localization capability: it can only indicate “inconsistency detected” without identifying which transaction caused the problem, forcing recovery to conservatively roll back all transactions within the snapshot

interval. Second, snapshot detection provides only coarse-grained diagnosis: it cannot distinguish whether errors originated in the scheduling layer (incorrect dependency relationships) or the execution layer (corrupted operations), complicating root cause analysis. These limitations motivate a complementary mechanism that operates at transaction-level granularity on the execution process.

Based on these considerations, we propose the second layer of our detection framework—transaction dependency verification based on transaction logs. This technique examines the execution process by analyzing and comparing transactions dependency relationships across replicas, detecting scheduling-layer errors caused by silent data corruption before they affect the final states. Unlike snapshot-based detection, dependency verification enables precise identification of specific transactions responsible for inconsistencies, allowing the system to roll back only affected transactions and their dependents during recovery rather than entire batches, thereby significantly improving fault tolerance efficiency. This transaction-level diagnostic capability effectively addresses the fault localization limitations of batch-level snapshot detection.

5.2. Transaction Dependency Analysis

Our deterministic database system assumes the existence of a global scheduler that coordinates transaction execution across replicas. This scheduler is responsible for: (1) analyzing read-write conflicts between transactions to determine logical dependency relationships, (2) ensuring that all replicas receive consistent dependency information, and (3) maintaining deterministic execution while allowing for different orders of independent concurrent transactions due to network latency and resource availability.

Under normal operation, the global scheduler guarantees that for identical transaction inputs, all replicas construct exactly the same dependency graph, regardless of differences in actual execution timing. Formally, let D_i denote the dependency graph constructed at replica i , then the scheduler ensures $D_1 = D_2 = \dots = D_n$ for all replicas. But independent concurrent transactions (those without dependency relationships) may execute in different orders across replicas. Our detection focuses on verifying dependency relationships—not execution order.

The scheduling process itself may be affected by SDC. Bit flips during read-write conflict analysis can cause incorrect dependency relationships.

Memory corruption in dependency graph data structures can lead to missing or incorrect dependency edges. Network transmission errors during distributed scheduling coordination can cause different replicas to receive different dependency information. These errors violate the consistency of the system.

In the analysis of transaction dependencies, if the write operations of a transaction T_1 depend on the read/write operations of another transaction T_2 —that is, T_2 arrives before T_1 , and both transactions access the same field—then T_1 is considered to depend on T_2 .

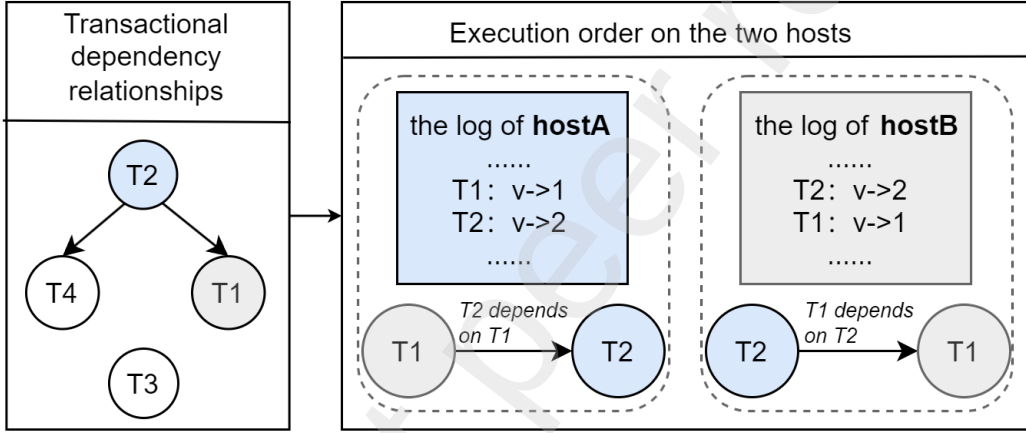


Figure 4: An example of a transaction dependency graph

Consider the following scenario in Figure 4 illustrating how scheduling corruption leads to detectable dependency mismatches (taking dual-host as an example):

Assuming there are two transactions: T_1 (write the value of the field v to 1), T_2 (write the value of the field to 2). The global scheduler correctly identifies that T_2 depends on T_1 due to the write-after-write dependency on field v . Both replicas receive this dependency information: $T_1 \rightarrow T_2$. But due to a bit flip in the dependency calculation algorithm on the coordination path to Host A, Host B incorrectly concludes that $T_2 \rightarrow T_1$ dependency, while Host A correctly maintains the $T_1 \rightarrow T_2$ dependency, leading to incorrect computation and eventual data inconsistency between replicas.

Theorem 1 (Error Equivalence). In a deterministic database system with global scheduling, a scheduling error S requires detection if and only if S eventually causes data inconsistency that affects query results.

Proof Sketch. If S leads to inconsistent query results, it must be detected to preserve correctness. Conversely, if S only affects scheduling metadata without altering data values or query outcomes, it is computationally irrelevant in a deterministic system. Such benign variations (e.g., minor timing differences that do not alter dependencies) are acceptable and need not be detected. $\square$

Corollary 1 (Detection Completeness). Dependency verification detects all scheduling errors threatening consistency, as such errors manifest as either dependency graph mismatches ($D_i \neq D_j$) or dependency constraint violations during execution.

This completeness property holds under the following conditions: (1) the system strictly adheres to a deterministic execution model; (2) the global scheduler correctly propagates dependency information to all replicas. Therefore, dependency relationship consistency serves as both a necessary and sufficient condition for detecting all scheduling errors that matter for system correctness. Based on this theoretical foundation, our detection algorithm implements dependency comparison as follows: Based on this theoretical foundation, our detection algorithm verifies dependency consistency through two phases: dependency graph construction from transaction logs, and cross-replica dependency comparison. If a previous block has modified the same field and appears earlier in the log, a dependency relationship is established between the current block and the previous one. The system records the set of transaction dependencies for each block. During the comparison phase, it compares the sets of dependent transaction IDs across hosts. If discrepancies are found, this indicates potential inconsistencies in the database state, which may lead to divergent user query results. Algorithm 3 presents the detailed procedure.

Lines 1–2: Initialize the iteration by traversing all transaction IDs present in the logs. Lines 3–4: For each transaction ID, retrieve the corresponding block sequences from logA and logB. Lines 5–10: Iterate over the blocks in order. For each block pair, obtain its dependency sets from the respective dependency graphs. Lines 11–13: If any dependency mismatch is detected, immediately return the error information specifying the transaction ID and block index. Line 14: If no mismatches are found after the traversal, return a message indicating that all transaction block dependencies are consistent.

For each transaction, the algorithm scans its accessed fields and checks for conflicts with previous transactions. Using a hash table to maintain field access records, each lookup operation is $O(1)$. For N transactions with an

Algorithm 3: Transaction Dependency Comparison Algorithm

Input: Two transaction logs $\log A$ and $\log B$ **Output:** Comparison result indicating whether dependency structures are consistent

```
1 foreach  $txn\_id \in AllTransactionIDs$  do
2    $blocks_A \leftarrow TxnToBlockMap_A[txn\_id]$ 
    $blocks_B \leftarrow TxnToBlockMap_B[txn\_id]$  /* Retrieve blocks
   from log A and log B */
3   for  $i \leftarrow 0$  to  $len(blocks_A) - 1$  do
4      $block\_id_A \leftarrow blocks_A[i]$   $block\_id_B \leftarrow blocks_B[i]$  /* Get
     block ID from log A and log B */
5      $deps_A \leftarrow DependencyGraph_A[block\_id_A]$ 
      $deps_B \leftarrow DependencyGraph_B[block\_id_B]$  /* Retrieve
     dependencies from log A and log B */
6     if  $deps_A \neq deps_B$  then
7       return Mismatch at transaction  $txn\_id$ , block index
        $(i + 1)$  /* Found inconsistency */
8     end
9   end
10 end
11 return All transaction block dependencies are consistent /* No
    inconsistencies found */
```

average of F fields accessed per transaction, the construction complexity is $O(N \cdot F)$. The algorithm iterates through all transaction IDs (let T denote the number of distinct transactions) and compares their dependency sets. Assuming each transaction has an average of D dependencies, the set comparison operation is $O(D)$ per transaction, leading to a total comparison complexity of $O(T \cdot D)$. Combining both phases, the total time complexity is $O(N \cdot F + T \cdot D)$. In practice, F and D are typically small constants (often less than 10), and $T \leq N$ (with equality when each transaction appears exactly once in the log). Therefore, the algorithm exhibits near-linear complexity $O(N)$ in the number of log entries.

6. Experiments

6.1. Experimental Setup

6.1.1. Experimental Environment

To simplify the setup of the experimental environment, reduce dependence on hardware resources, and validate the fundamental functionalities of the system in a relatively simple setting, this study employs a dual-thread approach to simulate a dual-machine architecture runtime environment.

Since the deterministic transaction execution system Aria is functionally incomplete, lacking many essential features of standard database systems and showing poor fault tolerance, this study retrieves transaction execution logs from Aria and performs simulation-based validation of the online detection mechanism.

6.1.2. Silent Data Corruption Injection

To simulate the occurrence of SDC, this study introduces random perturbations into certain field values within the transaction logs. Specifically, small disturbances (ranging from ± 1 to ± 3) are randomly applied to the hexadecimal representation of the target fields, typically value fields, to emulate errors caused by hardware faults such as memory bit flips or storage corruption during transaction execution. If the perturbed value becomes zero, the error is not injected to avoid producing invalid data. To control the frequency and distribution of injected faults, a parameter named `sdc_probability` is introduced, allowing precise regulation of the SDC injection probability. All perturbations injected by the system are recorded in detail, serving as the basis for subsequent evaluation of detection effectiveness and statistical analysis. Additionally, to simulate the effect of SDC on scheduling, which consequently alters inter-transaction dependencies, we randomly swap certain operations to deliberately disturb the original dependency relationships.

The system adopts TPC-C as the benchmark workload, which includes two types of transactions: `NewOrderQuery` and `MakeNewOrderQuery`, corresponding to order placement and payment operations, respectively. These transactions collectively cover nearly all table access patterns. The workload is divided into three types: `NEW_ORDER_ONLY` (containing only order transactions), `PAYMENT_ONLY` (containing only payment transactions), and `MIXED` (a balanced mix of both). At the commit stage of each transaction, its read and write sets are constructed. The write set, after being organized, serves as the input to the SDC detection mechanism.

Name	Description
Number of Partitions	Total number of partitions set during transaction generation; determines how many partitions are hashed during snapshot comparison.
SDC Injection Probability	Probability that a record is injected with a SDC, used to simulate fault conditions.
Error Detection Rate	Ratio of successfully detected SDC to the total number of injected errors.
Comparison Time	Time consumed to compare the results between hosts, excluding the time for snapshot construction.
Total Detection Time	Overall time taken by the SDC detection method.

Table 1: Experimental Parameters and Evaluation Metrics

6.1.3. Experimental Parameters and Metrics

This experiment involves several parameters, including the number of data partitions and the SDC injection rate. The evaluation metrics include the SDC detection rate and detection time, which reflect the efficiency of the proposed detection method. Detailed descriptions are shown in Table 1.

6.2. Experimental Results

The experimental evaluation of the proposed detection mechanism is conducted from three key perspectives: (1) assessing its intrinsic detection capability, (2) examining the influence of key parameter variations on detection results, and (3) comparing its performance with existing detection techniques.

6.2.1. Detection Performance of the Proposed Method

The experiment utilizes the TPC-C standard benchmark framework to generate mixed workloads, which are executed by the Aria system for validation. The number of partitions is set to 4, and the number of concurrent threads is set to 2. Each experiment group consists of 10 independently generated transaction datasets. SDC are injected using a randomized algorithm with an injection probability of 10^{-4} , and all injected errors are recorded for subsequent evaluation. To mitigate randomness, each configuration is tested five times, and the final performance metrics are reported as the arithmetic mean of the results.

Dataset ID	Log Entries	Detection Rate (%)	Comparison Time (s)	Total Time (s)
1	33,014	25	0.0093	0.3979
2	32,999	50	0.0103	0.3267
3	31,379	50	0.0085	0.2915
4	30,036	25	0.0057	0.3036
5	32,520	50	0.0053	0.2395
6	32,284	25	0.0084	0.2703
7	32,373	25	0.0064	0.3226
8	32,681	50	0.0057	0.2820
9	32,659	50	0.0102	0.2931
10	32,659	50	0.0080	0.3065

Table 2: Performance Evaluation of Incremental Snapshot Detection Technique

6.2.2. Performance of Snapshot-Based Incremental Detection

Table 2 presents the experimental evaluation results of the efficient snapshot validation technique based on a layered hash tree structure. The experiments were conducted using 10 different transaction log datasets, with each group injected with 3 SDC, corresponding to an approximate error injection rate of 0.01%. Each experiment was repeated 5 times to eliminate random noise, and the values reported in the table are the arithmetic means of the five measurements. The total detection time includes both the time to generate incremental snapshots and the time for comparison, representing the overall runtime of the algorithm.

The experimental results show that this technique can complete the state validation within subsecond time frames (0.2395-0.3979 s). However, during transaction execution, some SDC may be overwritten by subsequent write operations, resulting in a single-run detection rate ranging from 25% to 50%. Therefore, this layer of detection cannot capture all SDC. Since it performs real-time consistency checks during transaction execution and runs relatively frequently (potentially multiple times within a single checkpoint interval), it handles smaller volumes of data and enables the system to quickly identify and recover from errors, thus preventing the propagation of SDC. If inconsistencies are detected, errors can be confirmed immediately. However, if consistency is observed, further fine-grained checks are still required to ensure the correctness of transaction execution and consistency of database state.

Dataset ID	Log Entries	Detection Rate (%)	Detection Time (s)
1	70,015	100	1.9249
2	58,785	100	0.9503
3	51,643	100	1.0376
4	66,364	100	1.3674
5	61,550	100	1.0576
6	61,700	100	1.1065
7	70,846	100	1.7089
8	61,438	100	1.2608
9	58,785	100	1.1469
10	61,146	100	1.1527

Table 3: Performance Evaluation of Log-Based Transaction Dependency Detection Technique

6.2.3. Performance of Log-Based Transaction Dependency Detection Technique

Table 3 presents the experimental results of log-Based transaction dependency detection technique. In each experimental group, we randomly swap several entries to break transaction dependencies, thereby simulating the impact of silent errors on scheduling. Each group was executed five times, and the final result is the arithmetic mean of the five repetitions.

The experiments simulate the scenario of performing dependency-consistency detection after multiple batches of transaction executions but before data persistence. As a result, the log volume in each experimental group is relatively large. This detection process covers the full execution cycle of transactions and can achieve a 100% detection rate for SDC that cause inconsistencies in transaction dependencies during system scheduling. It is used to identify errors and locate the exact transactions involved. The two-level detection mechanisms complement each other, jointly ensuring the consistency of the database state.

6.3. Impact of Experimental Parameters on Detection Performance

This subsection focuses on analyzing the impact of the number of partitions and the SDC injection probability on the performance of the proposed method. The number of partitions directly determines how the partition field appears in the transaction log, thus affecting the method's ability to perform hash-based calculations and consistency verification at the subregion level.

As the number of partitions increases, the data is divided more granularly, which may influence both detection effectiveness and system performance. Evaluating the detection performance under different partition configurations helps assess the method's applicability and scalability across various deployment architectures. Meanwhile, the SDC injection probability, as a key parameter reflecting the potential fault rate in the system, affects both the frequency and distribution of errors in the transaction log. Investigating the method's detection accuracy and runtime performance under varying error injection probabilities is essential to verify its robustness and reliability in fault-prone or high-risk environments.

6.3.1. Impact of Partition Count on Error Detection

This study evaluates the performance of the proposed system through eight groups of experiments with different partition counts ($N = 2, 4, \dots, 16$). Each configuration was tested five times to mitigate the effects of randomness, and the arithmetic mean of the results is presented in Figure 5. The x-axis represents the number of partitions, while the y-axes show three key performance metrics: error detection rate, comparison time (ms), and total detection time (ms).

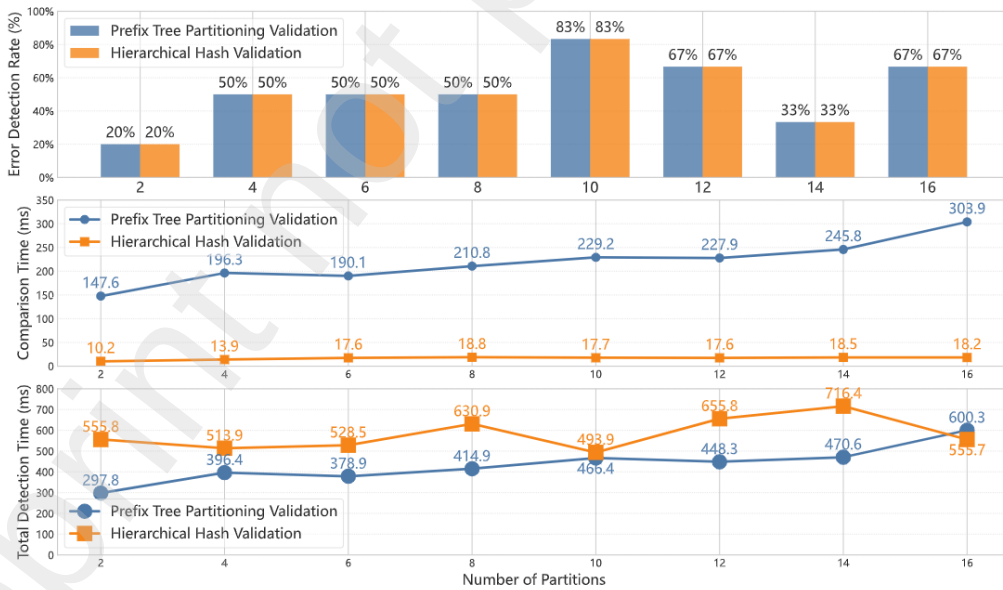


Figure 5: Impact of Partition Count on Detection Performance

Metric	Correlation Coefficient	p-value
Error Detection Rate (%)	Pearson $r = 0.477$	0.2323
Comparison Time (ms)	Spearman $\rho = 0.952$	0.0003
Total Detection Time (ms)	Spearman $\rho = 0.952$	0.0003

Table 4: Correlation Between Prefix Tree-Based Validation Performance and Partition Count

Metric	Correlation Coefficient	p-value
Error Detection Rate (%)	Pearson $r = 0.477$	0.2323
Comparison Time (ms)	Spearman $\rho = 0.683$	0.0621
Total Detection Time (ms)	Spearman $\rho = 0.416$	0.3051

Table 5: Correlation Between Hierarchical Hash Validation Performance and Partition Count

To further quantify the relationship between partition count and system performance, we conducted a correlation analysis. The results are shown in Tables 4 and 5, corresponding to the prefix tree-based validation and the hierarchical hash-based validation methods, respectively.

The results shown in Table 4 and Table 5 indicate that the number of partitions in both methods has no significant correlation with the error detection rate. This is consistent with theoretical expectations, as the locations of error injections are logically random and some errors may be overwritten by correct data. Since our method verifies all partitions, changes in partition granularity do not affect the overall detection capability. For the prefix tree partition validation method, the comparison time shows a significant positive correlation with the number of partitions, aligning with the algorithm’s theoretical complexity of $O(p)$. Due to the relatively simple construction and the relatively large proportion of comparison time in the total time, the prefix tree layered validation method also exhibits a notable correlation with the partition count. The hierarchical hash validation method’s comparison time shows a moderate positive correlation with partition number, consistent with the theoretical complexity of $O(\log p)$. However, the total detection time does not display a significant correlation with the partition count, mainly because it is affected by the snapshot generation latency, which is not closely related to partition number. This can be explained by the following:

$$T_{\text{total}} = T_{\text{snapshot}} + T_{\text{compare}} \approx c_1 + c_2 \log N \quad (2)$$

where T_{snapshot} is a system overhead that remains roughly constant, and although T_{compare} increases with the number of partitions, it accounts for a relatively small portion of the total time. Thus, the overall trend is not obvious. The performance of the incremental detection layer is mainly constrained by the underlying storage system.

6.3.2. Impact of Silent Data Corruption Injection Probability on Detection Performance

This section focuses on evaluating how different error injection rates affect the second-layer dependency-consistency detection. Since the incremental snapshot-based detection technique may miss errors that are subsequently overwritten by correct data, we do not include experiments for the first-layer method here. Here, we simulate the effect of silent errors on system scheduling by modifying the transaction dependency relationships. This set of experiments was conducted with the same dataset while varying the SDC injection rates at 0.5×10^{-4} , 1.0×10^{-4} , 1.5×10^{-4} , 2.0×10^{-4} , 2.5×10^{-4} , and 3.0×10^{-4} , totaling six groups of tests. Each group was repeated five times, and the average values were taken as the results.

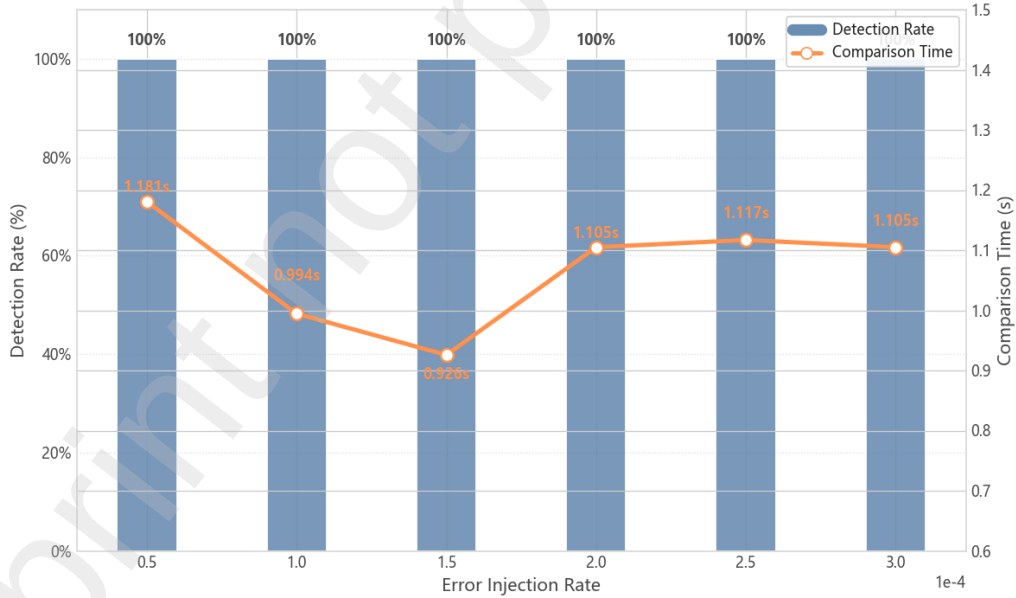


Figure 6: Impact of Error Injection Rate on the Performance of Error Detection Techniques

The experimental results, shown in Figure 6, plot the error injection rate

on the x-axis and the error detection rate on the y-axis. As the SDC injection rate increases, the detection rate remained at 100% across all experiments, indicating that the detection capability of the method is not affected by the error injection rate. This validates the robustness and reliability of the method under varying fault density scenarios. Moreover, the comparison time exhibited some irregular fluctuations at different injection rates without showing a significant correlation with the injection rate. This is mainly because the time overhead during the comparison process is largely influenced by system resource utilization (such as I/O load, CPU scheduling, etc.) and is not directly determined by the density of injected errors.

6.4. Comparison with Existing Methods

Furthermore, this paper compares two incremental snapshot-based detection technique with existing SDC detection methods RIVA [14] and a brute-force search method. RIVA detects SDC during data transmission by comparing secure hash checksums (e.g., SHA256) of source and target files, which only identifies whether errors have occurred but cannot locate or count the errors. Using the same 10 datasets with identical SDC injection rates, the four methods were tested with five repetitions per group, and averages were calculated. The results are presented in Figure 7, where the x-axis represents the dataset index, and the y-axes indicate error detection rate and total detection time (milliseconds), respectively.

As shown in the figure 7, the proposed methods can detect all SDC that are not later overwritten by correct data. In contrast, RIVA, which uses a whole-file hashing approach, can only detect the presence of errors but cannot identify their specific location or number, resulting in a significantly lower detection rate which is approximately 20% less than the proposed method. Regarding time efficiency, the proposed method demonstrates an average improvement of about 70.73% over RIVA. It is worth noting that the brute-force search method's average execution time exceeds 170 seconds, far surpassing the other methods, so it is not shown in the figure to preserve readability. Compared with the brute-force lookup, the proposed method matches its detection rate but improves time efficiency by approximately 80 times.

7. Conclusions

In this paper, we addressed the challenge of SDC in database systems running on general-purpose hardware. To overcome the limitations of ex-

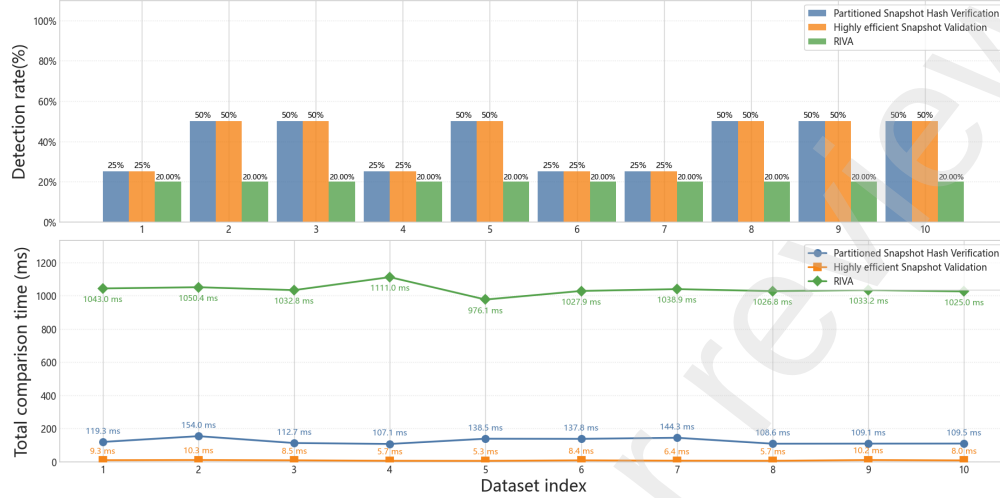


Figure 7: Comparison of three silent data corruption detection methods

isting hardware- and software-based approaches, we proposed a dual-layer software-level detection framework that combines snapshot-based incremental verification with asynchronous log-level consistency checks. This design ensures both timely detection during transaction execution and a final safeguard before data persistence. Experimental results demonstrate that our method achieves an approximately **20%** improvement in detection rate and a **70.83%** improvement in detection efficiency compared to traditional software-based solutions in transaction execution. Future work may explore integrating our framework with adaptive fault-tolerance strategies, extending it to distributed and cloud-native database systems. Such work could further improve the robustness and scalability of SDC detection in modern data management systems.

Acknowledgment

This work is supported by the National Natural Science Foundation of China (NSFC) (62232005, 62202126).

References

- [1] A. Singh, S. Chakravarty, G. Papadimitriou, and D. Gizopoulos, Silent data errors: Sources, detection, and modeling, in *Proceedings of the IEEE 41st VLSI Test Symposium (VTS)*, 2023, pp. 1–12.

- [2] H. D. Dixit, S. Pendharkar, M. Beadon, C. Mason, T. Chakravarthy, B. Muthiah, and S. Sankar, Silent data corruptions at scale, *arXiv preprint arXiv:2102.11245*, 2021.
- [3] 51CTO Technical Community, Silent data corruption: How Oracle Database addresses and handles it, 2018. [Online]. Available: <https://www.51cto.com/article/580933.html>
- [4] Alibaba Cloud, Elastic Compute Service exception report, 2022. [Online]. Available: https://help.aliyun.com/document_detail/405251.html
- [5] Google Cloud, Reliability white paper: When good servers go bad, 2021. [Online]. Available: <https://cloud.google.com/blog/products/infrastructure/availability-part-3-when-good-servers-go-bad>
- [6] R. Koller *et al.*, When storage systems fail, in *Proceedings of the USENIX Conference on File and Storage Technologies (FAST)*, 2020.
- [7] J. Kreps, N. Narkhede, and J. Rao, Kafka: A distributed messaging system for log processing, in *Proceedings of NetDB*, 2011.
- [8] Y. Lu, X. Yu, L. Cao, and S. Madden, Aria: A fast and practical deterministic OLTP database, *Proceedings of the VLDB Endowment*, 2020.
- [9] A. Thomson, T. Diamond, S.-C. Weng, K. Ren, P. Shao, and D. J. Abadi, Calvin: Fast distributed transactions for partitioned database systems, in *Proceedings of the ACM SIGMOD International Conference on Management of Data*, 2012.
- [10] Tencent Cloud Developer Community, Silent errors: How does the Oracle Database respond and handle them?, 2018. [Online]. Available: <https://cloud.tencent.com/developer/article/1189307>
- [11] R. Makaram and K. S. Yap, System, apparatus and method for providing protection against silent data corruption in a link, U.S. Patent 12,164,371, Dec. 10, 2024.
- [12] B. Zhang, S. Ainsworth, L. Mukhanov, and T. M. Jones, Parallaft: Runtime-Based CPU fault tolerance via heterogeneous parallelism, in

Proceedings of the 23rd ACM/IEEE International Symposium on Code Generation and Optimization, 2025, pp. 584–599.

- [13] R. Dutta, H. D. Dixit, R. Van Riel, G. Vunnam, and S. Sankar, Hardware Sentinel: Protecting software applications from hardware silent data corruptions, in *Proceedings of the 30th ACM International Conference on Architectural Support for Programming Languages and Operating Systems (Volume 2)*, 2025, pp. 482–497.
- [14] B. Charyyev, A. Alhussen, H. Sapkota, E. Pouyoul, M. H. Gunes, and E. Arslan, Towards securing data transfers against silent data corruption, in *Proceedings of the 19th IEEE/ACM International Symposium on Cluster, Cloud and Grid Computing (CCGRID)*, 2019, pp. 262–271.
- [15] N. Yang and Y. Wang, Identify silent data corruption vulnerable instructions using SVM, *IEEE Access*, vol. 7, pp. 40210–40219, 2019.
- [16] E. Berrocal, L. Bautista-Gomez, S. Di, Z. Lan, and F. Cappello, Toward general software-level silent data corruption detection for parallel applications, *IEEE Transactions on Parallel and Distributed Systems*, vol. 28, no. 12, pp. 3642–3655, 2017.
- [17] J. Li, Z. Shen, and X. Meng, Scientific big data management: Concepts, technologies, and systems, *Journal of Computer Research and Development*, vol. 54, no. 2, pp. 235–247, 2017. (in Chinese)
- [18] Y. Zhang, X. Wang, X. Liu, and L. Liu, A survey on security in cloud computing environments, *Journal of Software*, vol. 27, no. 6, pp. 1328–1348, 2016. (in Chinese)
- [19] D. J. Abadi and J. M. Faleiro, An overview of deterministic database systems, *Communications of the ACM*, vol. 61, no. 9, pp. 80–89, 2018.
- [20] C. Borchert, H. Schirmeier, and O. Spinczyk, Compiler-implemented differential checksums: Effective detection and correction of transient and permanent memory errors, in *Proceedings of the 53rd IEEE/IFIP International Conference on Dependable Systems and Networks (DSN)*, 2023, pp. 81–94.

- [21] K. Joshi, R. Singh, T. Bassetto, S. V. Adve, D. Marinov, and S. Misailovic, FastFlip: Compositional error injection analysis, *CoRR*, vol. abs/2403.13989, 2024.
- [22] M. J. Liao, S. Ainsworth, L. Mukhanov, A. Barredo, M. Kynigos, and T. M. Jones, ParaVerser: Harnessing heterogeneous parallelism for affordable fault detection in data centers, in *Proceedings of the 55th IEEE/IFIP International Conference on Dependable Systems and Networks (DSN)*, 2025, pp. 386–400.
- [23] J. Ma, H. Pei, L. Lausen, and G. Karypis, Understanding silent data corruption in LLM training, in *Proceedings of the 63rd Annual Meeting of the Association for Computational Linguistics (ACL)*, 2025.
- [24] Y. Jiang, Z. Zhou, B. Xu, B. Liu, R. Xu, and P. Huang, Training with confidence: Catching silent errors in deep learning training with automated proactive checks, *arXiv preprint arXiv:2506.14813*, 2025.
- [25] Y. Liang, Y. AuthorTwo, and Y. AuthorThree, ATTNChecker: Algorithm-based fault tolerance for transformer attention, in *Proceedings of the ACM SIGPLAN Symposium on Principles and Practice of Parallel Programming (PPoPP)*, 2025.
- [26] M. Yakhchi, M. Fazeli, and S. Asghari, Silent data corruption estimation and mitigation without fault injection, *IEEE Canadian Journal of Electrical and Computer Engineering*, vol. 45, no. 3, pp. 318–327, 2022.
- [27] P. Vimalachandran, H. Wang, Y. Zhang, B. Heyward, and F. Whitaker, Ensuring data integrity in electronic health records: A quality health care implication, in *Proceedings of the International Conference on Orange Technologies (ICOT)*, 2016, pp. 20–27.
- [28] S. Greca and A. Kosta, The impact of the Google Cloud to increase the performance for a use case in e-commerce platform, *International Journal of Computer Applications*, 2020.
- [29] W. Chen, Reliability optimization analysis of enterprise financial management system using the Markov model, *Mobile Information Systems*, vol. 2022, Article ID 4829902, 2022.

- [30] J. Gray, The transaction concept: Virtues and limitations, in *Proceedings of the 7th International Conference on Very Large Data Bases*, 1981, pp. 144–154.
- [31] C. Mohan, D. Haderle, B. Lindsay, H. Pirahesh, and P. Schwarz, ARIES: A transaction recovery method supporting fine-granularity locking and partial rollbacks using write-ahead logging, *ACM Transactions on Database Systems*, vol. 17, no. 1, pp. 94–162, 1992.
- [32] P. A. Bernstein, V. Hadzilacos, and N. Goodman, *Concurrency Control and Recovery in Database Systems*, Addison-Wesley, 1987.
- [33] E. O’Neil, P. O’Neil, D. Gawlick, and E. O’Neil, The log-structured merge-tree, *Acta Informatica*, vol. 33, no. 4, pp. 351–385, 1996.
- [34] F. Chang, J. Dean, S. Ghemawat, W. C. Hsieh, D. A. Wallach, M. Burrows, T. Chandra, A. Fikes, and R. E. Gruber, Bigtable: A distributed storage system for structured data, *ACM Transactions on Computer Systems*, vol. 26, no. 2, pp. 1–26, 2008.
- [35] D. Ongaro and J. Ousterhout, In search of an understandable consensus algorithm, in *Proceedings of the USENIX Annual Technical Conference*, 2014, pp. 305–319.
- [36] D. Comer, Ubiquitous B-tree, *ACM Computing Surveys*, vol. 11, no. 2, pp. 121–137, 1979.
- [37] J. Geng, H. Wang, and Y. Yan, EMIT: Micro-invasive database configuration tuning, *arXiv preprint arXiv:2406.00616*, 2024.

